



# SISTEMAS OPERATIVOS - TEORÍA

14 de Junio de 2024

Nombre \_\_\_\_\_ DNI \_\_\_\_\_  
Apellidos \_\_\_\_\_ Grupo \_\_\_\_\_

**Cuestión 1 (2 puntos).** Un sistema empujado de una videoconsola portátil dispone de un sistema de ficheros en flash de bloques indexados tipo UNIX, pero en el que la estructura de los nodos-i se ha simplificado, incluyendo solamente una entrada directa del índice y una indirecta simple. El tamaño de bloque usado por el sistema es tan sólo de 16 bytes y usa direcciones de bloque de 1 Byte. A continuación se muestran las estructuras principales del sistema de ficheros en un instante dado, incluyendo la máscara de bits, parte de la tabla de nodos-i (los que están usados) y el contenido de los bloques referenciados por estos nodos-i.

Máscara de bits: 1 0 0 1 0 1 1 1 0 0 0 0 1 0 0 1 0 0 ..... 0

nodo-i 2                      nodo-i 3                      nodo-i 4                      nodo-i 5                      nodo-i 9

|           |      |           |     |           |      |           |      |           |      |
|-----------|------|-----------|-----|-----------|------|-----------|------|-----------|------|
| Tam       | 1    | Tam       | 1   | Tam       | 1    | Tam       | 1    | Tam       | 1    |
| #enlaces  | 4    | #enlaces  | 1   | #enlaces  | 1    | #enlaces  | 2    | #enlaces  | 2    |
| Tipo F/D  | D    | Tipo F/D  | F   | Tipo F/D  | F    | Tipo F/D  | D    | Tipo F/D  | D    |
| Permisos  | rwX  | Permisos  | rw- | Permisos  | rw-  | Permisos  | r-x  | Permisos  | rwX  |
| Directo   | 3    | Directo   | 6   | Directo   | 12   | Directo   | 0    | Directo   | 5    |
| Indirecto | Null | Indirecto | 7   | Indirecto | Null | Indirecto | Null | Indirecto | Null |

Bloque 0                      Bloque 3                      Bloque 5                      Bloque 6                      Bloque 7                      Bloque 12                      Bloque 15

|    |   |    |   |    |   |      |    |      |      |
|----|---|----|---|----|---|------|----|------|------|
| .  | 5 | .  | 2 | .  | 9 | 1234 | 15 | 4321 | 1111 |
| .. | 2 | .. | 2 | .. | 5 | 1234 |    | 4321 | 2222 |
| D  | 4 | A  | 3 | E  | 4 | 1234 |    | 4321 | 3333 |
|    |   |    |   |    |   | 1234 |    | 4321 | 4444 |
|    |   | B  | 5 |    |   |      |    |      |      |
|    |   | C  | 9 |    |   |      |    |      |      |

En el sistema uno de los procesos ejecuta el siguiente programa. Responde a las siguientes preguntas:

```
int main( void ) {  
    char buf[16] = {[0 ... 15] = '0'};  
    int fd1,fd2,n;  
    1: fd1 = open("/C/E", O_RDWR);  
    2: fd2 = open("/B/D", O_RDWR);  
    3: lseek(fd1,64,SEEK_CUR);  
    4: write(fd1,buf,16);  
    5: write(fd2,buf,8);  
    6: n = read(fd2,buf,8);  
    7: n = read(fd1,buf,8);  
    8: n = read(fd2,buf,16);  
    9: close(fd1);  
    10: close(fd2);  
}
```

- (0.75 puntos)** Dibuja el árbol de directorios inicial. Si hay alguna inconsistencia en las estructuras mostradas arriba, indica cómo corregirla.
- (0.75 puntos)** ¿Qué bloque lógico del fichero /C/E se ha escrito al completarse la operación *write()* de la línea 4? Explica los cambios que esta escritura ha supuesto en las estructuras del sistema de ficheros mostradas arriba.
- (0.5 puntos)** ¿Qué valores toma la variable *n* tras cada operación de read (líneas 6-8)? Razona la respuesta.



**Cuestión 2 (2 puntos).** Considere un sistema monoprocesador donde se lanzan 4 tareas monohilo (de T1 a T4). La siguiente tabla indica el tiempo de llegada de cada tarea, su valor de prioridad (cuanto menor número, mayor prioridad), así como su perfil de ejecución (tiempo requerido para ráfagas de CPU y fases de entrada-salida alternadas):

| Tarea | Prioridad | Llegada | CPU | E/S | CPU | E/S |
|-------|-----------|---------|-----|-----|-----|-----|
| T1    | 4         | 0       | 5   | 1   | 1   | 1   |
| T2    | 3         | 1       | 3   | 1   | 1   |     |
| T3    | 2         | 2       | 1   | 1   | 1   | 1   |
| T4    | 1         | 3       | 2   | 1   | 2   |     |

- a) Rellene el diagrama de planificación mostrado más abajo, asumiendo que el planificador de procesos del sistema operativo usa una política de planificación expropiativa basada en prioridades. Se ha de proporcionar también el tiempo de ejecución y de espera de cada proceso, así como la productividad y el porcentaje de uso de CPU. Al completar el diagrama, se debe emplear la siguiente notación para representar el estado de las tareas:

- *En ejecución*: rellenar con “X”
- *Bloqueado por E/S*: rellenar con “O”
- *Listo para ejecutar*: rellenar con “--”

**Nota importante:** En caso de que suceda algún conflicto de planificación (motivado porque dos tareas puedan encolarse al mismo tiempo en la *run queue*) se ha de asumir que el sistema da preferencia a las tareas que tienen un menor número frente a tareas con mayor número (p.ej., T1 se insertaría en la cola antes que T2, T3 o T4, y así sucesivamente)

|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|
| T1 |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| T2 |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| T3 |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| T4 |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |

|                  | T1 | T2 | T3 | T4 |
|------------------|----|----|----|----|
| Tiempo de ejec.  |    |    |    |    |
| Tiempo de espera |    |    |    |    |

Productividad: \_\_\_\_\_

Porcentaje de uso de CPU: \_\_\_\_\_

- b) Rellene el diagrama de planificación mostrado más abajo, asumiendo que el planificador de procesos del sistema operativo usa una política de planificación Round Robin con cuanto 2 para distribuir ciclos de CPU.

|    | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|----|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|
| T1 |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| T2 |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| T3 |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |
| T4 |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |    |    |    |    |

|                  | T1 | T2 | T3 | T4 |
|------------------|----|----|----|----|
| Tiempo de ejec.  |    |    |    |    |
| Tiempo de espera |    |    |    |    |

Productividad: \_\_\_\_\_

Porcentaje de uso de CPU: \_\_\_\_\_

**Cuestión 3 (2 puntos)** Considere el siguiente programa, que se ejecuta en un sistema GNU/Linux.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>#include &lt;pthread.h&gt; #include &lt;stdio.h&gt; #include &lt;fcntl.h&gt; #include &lt;string.h&gt; #include &lt;unistd.h&gt;  #define NR_THREADS 4 #define MYBUFLLEN 10  pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER; char* tokens[NR_THREADS]={ "aaaa", "bbbb",                            "cccc", "dddd"};  int pos = 0; int fd1 = -1; int fd2 = -1;  void* threadfun(void * arg){     int chunk;     int len;     char buf1[MYBUFLLEN];     const char* buf2;     int id=(int) arg;      pthread_mutex_lock(&amp;lock);     chunk=pos++;     len=read(fd1,buf1,MYBUFLLEN-1);     buf2=tokens[chunk];     write(fd2,buf2,strlen(buf2));     pthread_mutex_unlock(&amp;lock);     buf1[len]='\0';     printf("Thread %d read %s\n",id,buf1);     return NULL; }</pre> | <pre>int main(void){     int i=0;     pthread_t desc[NR_THREADS];     char buf[MYBUFLLEN];     int nr_bytes=0;      fd2=open("a.txt",O_WRONLY O_CREAT O_TRUNC, 0666);     fd1=open("a.txt",O_RDONLY);      for (i = 0; i &lt; NR_THREADS; i++ )         pthread_create(&amp;desc[i],NULL,                       threadfun,                       (void*)i);      for (i = 0; i &lt; NR_THREADS; i++ )         pthread_join(desc[i],NULL);      lseek(fd1,0, SEEK_SET);      while((nr_bytes=read(fd1,buf,MYBUFLLEN))&gt;0)         write(1,buf,nr_bytes);      write(1,"\n",1);      close(fd1);     close(fd2);      return 0; }</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- a) ¿Podría fallar alguna de las dos llamadas a `open()` en el programa, suponiendo que el usuario propietario del proceso que ejecuta este programa tiene permisos de escritura en el directorio de trabajo del proceso? Justifique la respuesta.

- b) Indique de forma justificada si la salida del programa y el contenido final del fichero "a.txt" pueden variar en distintas ejecuciones del mismo. Proporcione una posible salida, y un posible contenido final del fichero

- c) Cada hilo del programa se encarga de escribir en el fichero una cadena de caracteres almacenada en el array tokens, siendo ésta distinta para cada hilo. Indique si en distintas ejecuciones un mismo hilo del programa (hilo 1, 2, etc) escribe siempre la misma cadena de caracteres (miembro del array tokens) en el fichero y en la misma posición. Justifique su respuesta. En caso negativo, discuta qué modificaciones sería necesario realizar en el programa (especificando el código correspondiente) para que cada hilo escribiese siempre la misma cadena del array tokens y en la misma posición del fichero, sin usar mecanismos de sincronización adicionales.

**Cuestión 4 (2 puntos)** Considera una implementación alternativa del problema del productor consumidor que permita tener dos tipos de productores con distinta prioridad de inserción. El esquema de código del productor y del consumidor sería el siguiente:

|                                                                                                                                                                                                                |                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>#define N 4 int buf[N];  int produce(void); void insert(int elem, int prio);  void producer(int prio) {     int elem;     while (1) {         elem = produce();         insert(elem, prio);     } }</pre> | <pre>void consume(int elem); int extract(void);  void consumer(void) {     int elem;     while (1) {         elem = extract();         consume(elem);     } }</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Asumiendo que *producer()* y *consumer()* son funciones invocadas por las funciones asociadas a cada hilo POSIX productor y consumidor respectivamente, proporciona una implementación válida de las funciones *insert()* y *extract()*, acordes con los prototipos indicados arriba, que permitan la sincronización correcta de estos hilos de acuerdo a las siguientes restricciones:

- Los productores con **prio 0** pueden insertar en el buffer siempre que haya un hueco libre.
- Los productores con **prio 1** sólo pueden insertar en el buffer si no hay ningún productor con prio 0 esperando y hay un hueco libre.

Utiliza mutex y variables de condición o semáforos para sincronizar los hilos y añade las variables globales necesarias.



**Cuestión 5 (2 puntos)** El siguiente programa se utiliza para comparar dos ficheros de 10 kB en un sistema tipo UNIX que usa páginas de 10 kB. El código se compila y enlaza estáticamente de modo que la biblioteca añade 1 página de datos y otra de código. El sistema inicializa el proceso con 2 kB de pila.

```
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

struct ThreadArgs {
    char *org1;
    char *org2;
    int tam;
};
int result;

int *compare_buffers(void *args) {
    struct ThreadArgs *thread_args = (struct ThreadArgs *)args;
    // C
    if (memcmp(thread_args->org1, thread_args->org2, thread_args->tam) == 0)
        result = 1; // Files are equal
    else
        result = 0; // Files are different
}

int main(int argc, char **argv) {
    int fd1, fd2, tam, result = 0;
    char *org1, *org2;
    struct stat bstat;
    pthread_t thread;
    struct ThreadArgs thread_args;

    if (argc != 3) {
        fprintf(stderr, "Usage: %s file1 file2\n", argv[0]);
        return 2;
    }

    if ((fd1 = open(argv[1], O_RDONLY)) < 0)
        err(EXIT_FAILURE, "Error opening the file");
    if ((fd2 = open(argv[2], O_RDONLY)) < 0)
        err(EXIT_FAILURE, "Error opening the file");
    if (fstat(fd1, &bstat) < 0)
        err(EXIT_FAILURE, "Error calling fstat");
    tam = bstat.st_size;
    if (fstat(fd2, &bstat) < 0)
        err(EXIT_FAILURE, "Error calling fstat");
    // A
    if (tam == bstat.st_size) {
        if ((org1 = mmap(NULL, tam, PROT_READ, MAP_PRIVATE, fd1, 0)) == MAP_FAILED)
            err(EXIT_FAILURE, "Error mapping the file");
        if ((org2 = mmap(NULL, tam, PROT_READ, MAP_PRIVATE, fd2, 0)) == MAP_FAILED)
            err(EXIT_FAILURE, "Error mapping the file");
        thread_args.org1 = org1;
        thread_args.org2 = org2;
        thread_args.tam = tam;
        // B
        pthread_create(&thread, NULL, compare_buffers, &thread_args);
        pthread_join(thread, NULL);
        // D
        munmap(org1, tam);
        munmap(org2, tam);
    }
    if (result)
        printf("Files are equal\n");
    else
        printf("Files are different\n");
    // E
    close(fd1);
    close(fd2);
    return result;
}
```

Responda a las siguientes preguntas:

- a) **(0.5 puntos)** Complete la siguiente tabla, indicando para cada símbolo la región de memoria en la que aparece en el mapa de memoria de ambos procesos, la protección de dicha región y si tiene soporte en el fichero ejecutable o no lo tiene:

| <i>Símbolo</i> | <i>Región de Memoria</i> | <i>Protección (RWX)</i> | <i>Soporte en Ejecutable (si/no)</i> |
|----------------|--------------------------|-------------------------|--------------------------------------|
| thread_args    |                          |                         |                                      |
| org1           |                          |                         |                                      |
| *org1          |                          |                         |                                      |
| stderr         |                          |                         |                                      |

- b) **(1 punto)** Describe las regiones de memoria del espacio de direcciones del proceso en el punto marcado con el comentario // **A**. A partir de ahí, justifica los cambios que va experimentando dicho espacio de direcciones en los puntos marcados con los comentarios // **B**, // **C**, // **D** y // **E**, justificando el motivo de la aparición y desaparición de regiones.

- c) **(0.5 puntos)** Justifica cuántos fallos de página se producen entre el punto // **A** y el punto // **E** e indique en qué partes del código se producen.